



PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS
GRADUAÇÃO EM ENGENHARIA DE SOFTWARE

Arquitetura de Software

Aula 12 – *Trade-offs* Arquiteturais

Prof.: Filipe Nhimi

Agenda

- ❑ Conceitos
- ❑ Exemplos
- ❑ Tipos de *Trade-offs*
- ❑ Teorema CAP e PACELC
- ❑ Conclusão



Viajando com **economia**
máxima de combustível



Viajando com economia
satisfatória de combustível e
um pouco mais confortável



Viajando **sem economia**
de combustível,
confortável e seguro



Viajando de forma
econômica,
“confortável”, seguro e
dormindo



Viajando de forma
“econômica”,
“confortável”, seguro,
dormindo e rápido



Viajando de forma
“econômica”,
“confortável”, seguro,
dormindo e rápido



É possível ter um sistema, ao mesmo tempo:

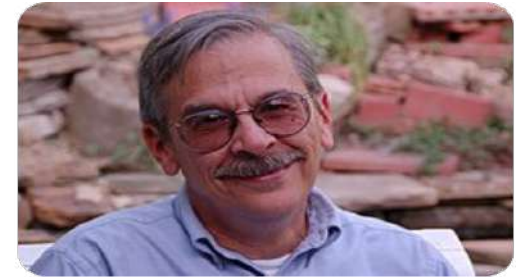
- Extremamente rápido
- Extremamente seguro
- Altamente disponível
- Totalmente consistente



choice

O que são *Trade-offs* Arquiteturais?

“Segundo Bass, Clements e Kazman, *trade-offs* são as **consequências inevitáveis** das decisões arquiteturais, nas quais a **melhoria de um atributo** de qualidade geralmente ocorre à custa da **degradação de outro**.



Len Bass



Paul Clements



Rick Kazman

Toda decisão
arquitetural **resolve um**
problema... e cria outro.

**Não existe arquitetura perfeita, apenas
escolhas bem justificadas.**

Trade-offs são **decisões**
arquiteturais que envolvem
concessões, onde ao melhorar
um aspecto do sistema, **outro pode**
ser impactado negativamente.

RAS → Direcionam decisões

Decisões → Geram *trade-offs*

Exemplo de *Trade-offs*

Desempenho vs Consistência



• 2nd

| Backend .NET | ASP.NET Core, C#, SQL Server, Azure | APIs RE...

4d •

+ Follow

"Reduzi de ~3000ms (3 segundos) para praticamente 0ms... mas percebi um problema importante."

Estudando arquitetura em APIs com .NET, implementei cache em memória usando **IMemoryCache** na camada de service.

O ganho de performance foi claro:

primeira requisição → ~3000ms (simulando acesso ao banco)

próximas requisições → ~0ms com cache

Exemplo de *Trade-offs*

Desempenho vs Consistência

Mas aí veio :
e quando os dados mudam?

Sem tratamento, o cache pode servir dados desatualizados.
Foi aí que implementei invalidação de cache.

Sempre que um novo produto é criado, removo o cache com:
`_cache.Remove("produtos");`

Assim, a próxima requisição força a atualização dos dados e mantém consistência.

Exemplo de *Trade-offs*

Desempenho vs Consistência

Resumo do fluxo agora:

busca no cache primeiro

se não existir → consulta banco + salva no cache

ao alterar dados → invalida o cache

Isso me fez entender que performance sem consistência não resolve o problema completo.

Exemplo de *Trade-offs*

Desempenho vs Consistência



Filipe Nhimi • You

9h ...

Diretor de Tecnologia | Professor de Engenharia de Software na PUC...

É exatamente assim que funciona a Arquitetura de Software: você resolve um problema e encontra outro pelo caminho. E agora?

Tomei uma decisão arquitetural de usar cache não distribuído para ganhar desempenho e, como pedágio, recebo o problema da consistência. O que fazer?

Entenda o cenário e discuta com stakeholders próximos desse domínio de negócio. Compreenda os trade-offs arquiteturais como escolhas.

Exemplo de *Trade-offs*

Desempenho vs Consistência

Talvez o contexto permita conviver bem com consistência eventual para obter melhor desempenho. Cada domínio de negócio lida com isso de forma diferente.

Um serviço distribuído de saldo bancário, por exemplo, dificilmente admitiria consistência eventual e, por isso, acaba abrindo mão de parte da disponibilidade.

Não existem receitas de bolo. O desafio está em encontrar o equilíbrio.

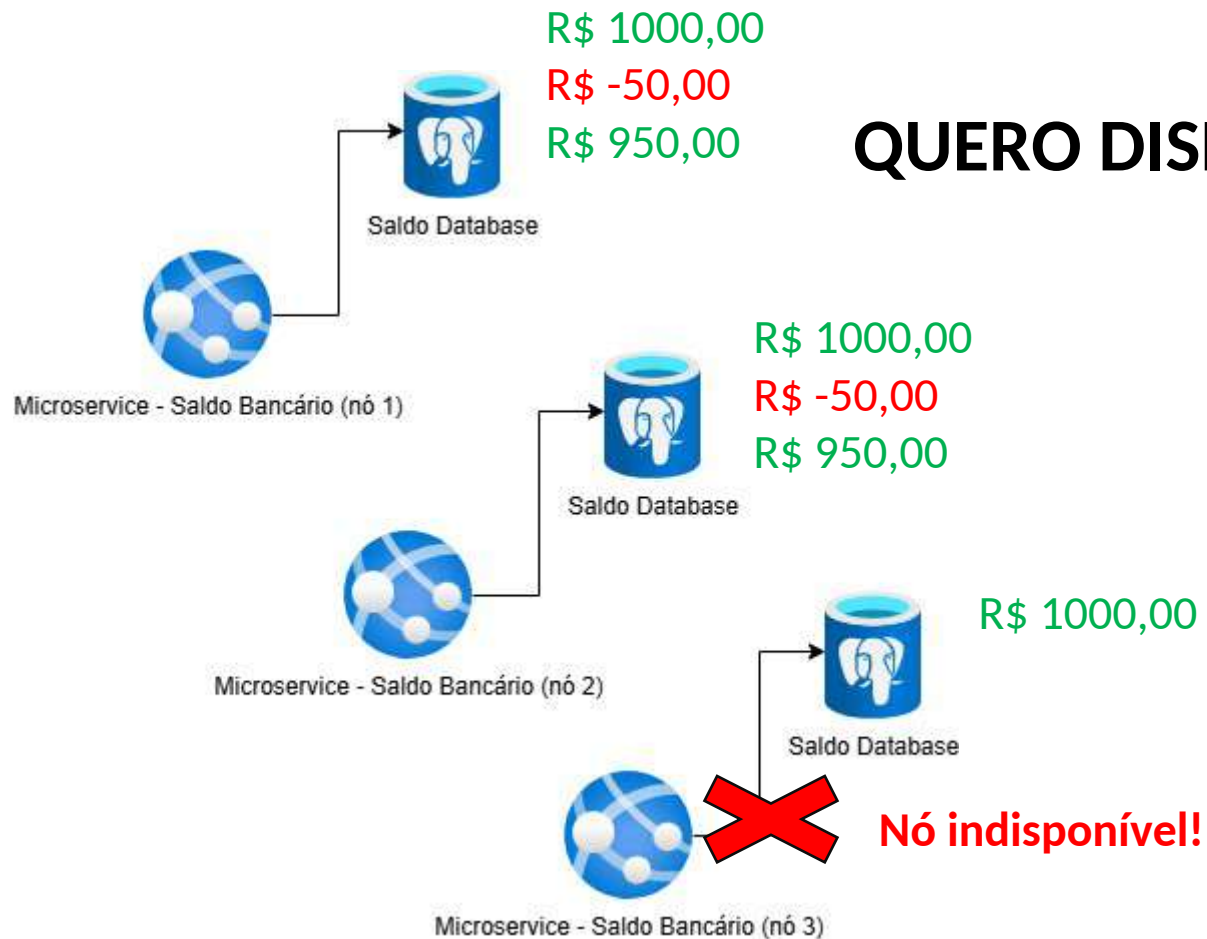
Exemplo de *Trade-offs*

Segurança vs Usabilidade



Exemplo de *Trade-offs*

Disponibilidade vs Consistência



QUERO DISPONIBILIDADE E CONSISTÊNCIA!



Qual o meu saldo?

R\$ 1000,00

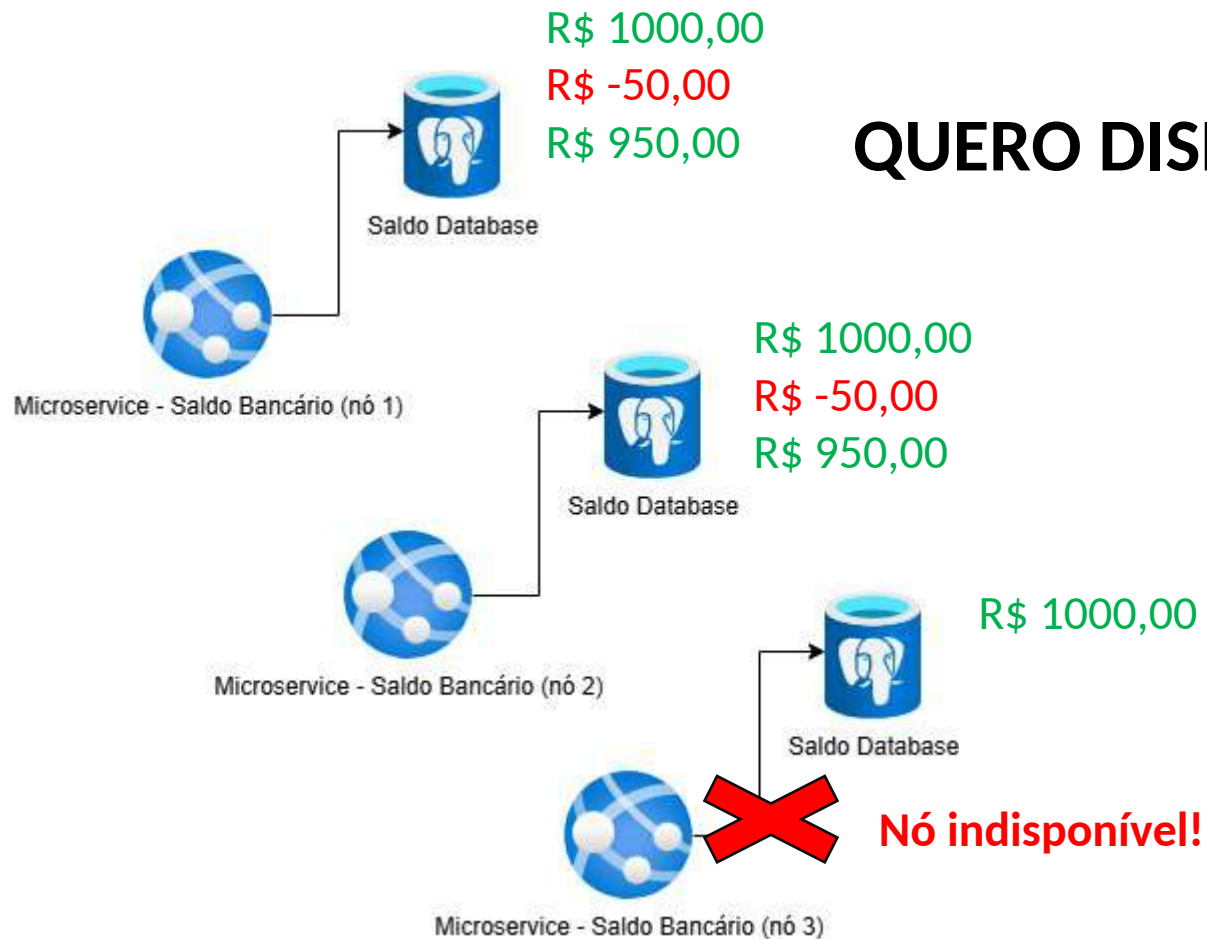
Faça um Pix de R\$ 50,00

Qual o meu saldo?

R\$ 950,00 (por sorte,
a requisição chegou na
instância de nó 2)

Exemplo de *Trade-offs*

Disponibilidade vs Consistência



QUERO DISPONIBILIDADE E CONSISTÊNCIA!



Qual o meu saldo?

R\$ 1000,00

Faça um Pix de R\$ 50,00

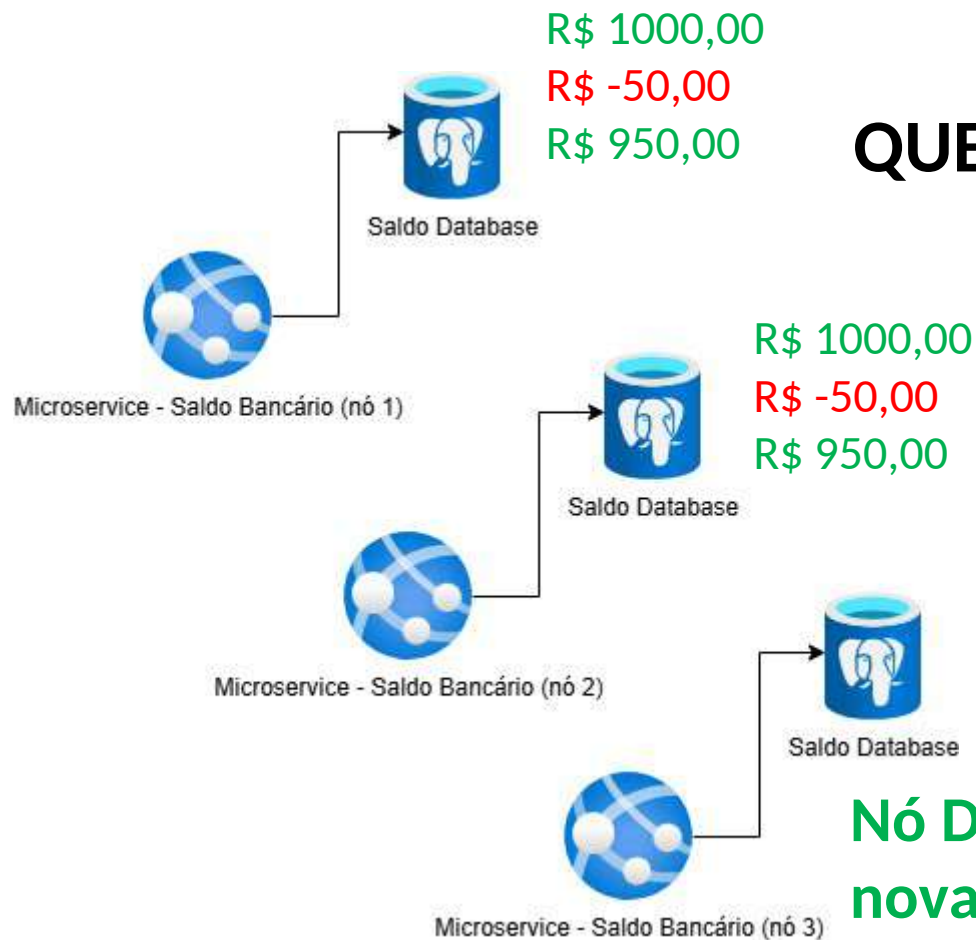
Qual o meu saldo?

R\$ 950,00 (por sorte,
a requisição chegou na
instância de nó 2)

Faça um Pix de R\$ 1000,00

Exemplo de *Trade-offs*

Disponibilidade vs Consistência ✖



QUERO DISPONIBILIDADE E CONSISTÊNCIA!



Qual o meu saldo?

R\$ 1000,00

Faça um Pix de R\$ 50,00

Qual o meu saldo?

R\$ 950,00 (por sorte,
a requisição chegou na
instância de nó 2)

Faça um Pix de R\$ 1000,00

Qual o meu saldo?

R\$ 1000,00 (pix liberado)

Sistema distribuído: Escolha entre Consistência (C) **ou** Disponibilidade (A)

Exemplo de *Trade-offs*

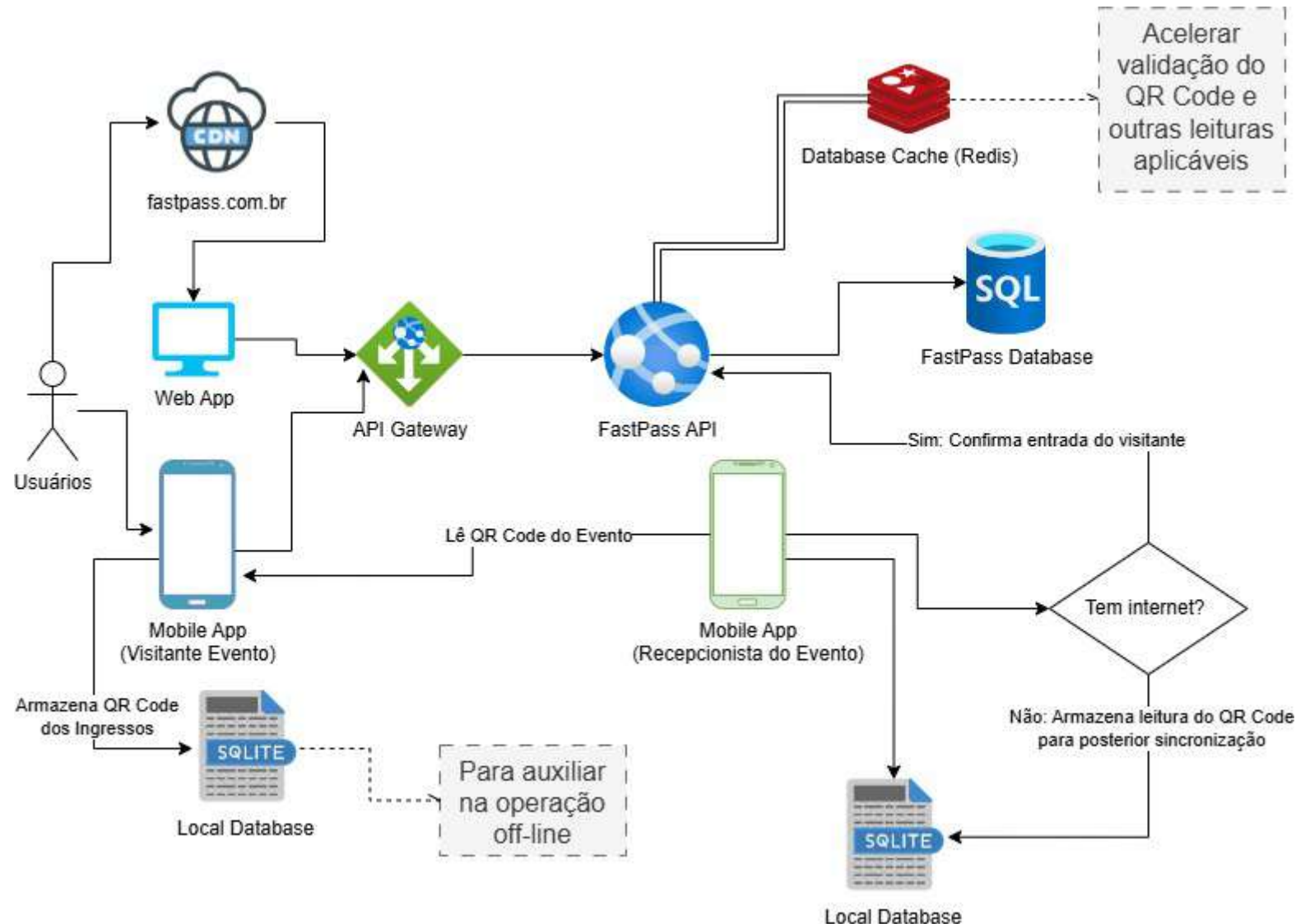
Disponibilidade vs Consistência

Outras situações: Desafio Técnico do FastPass

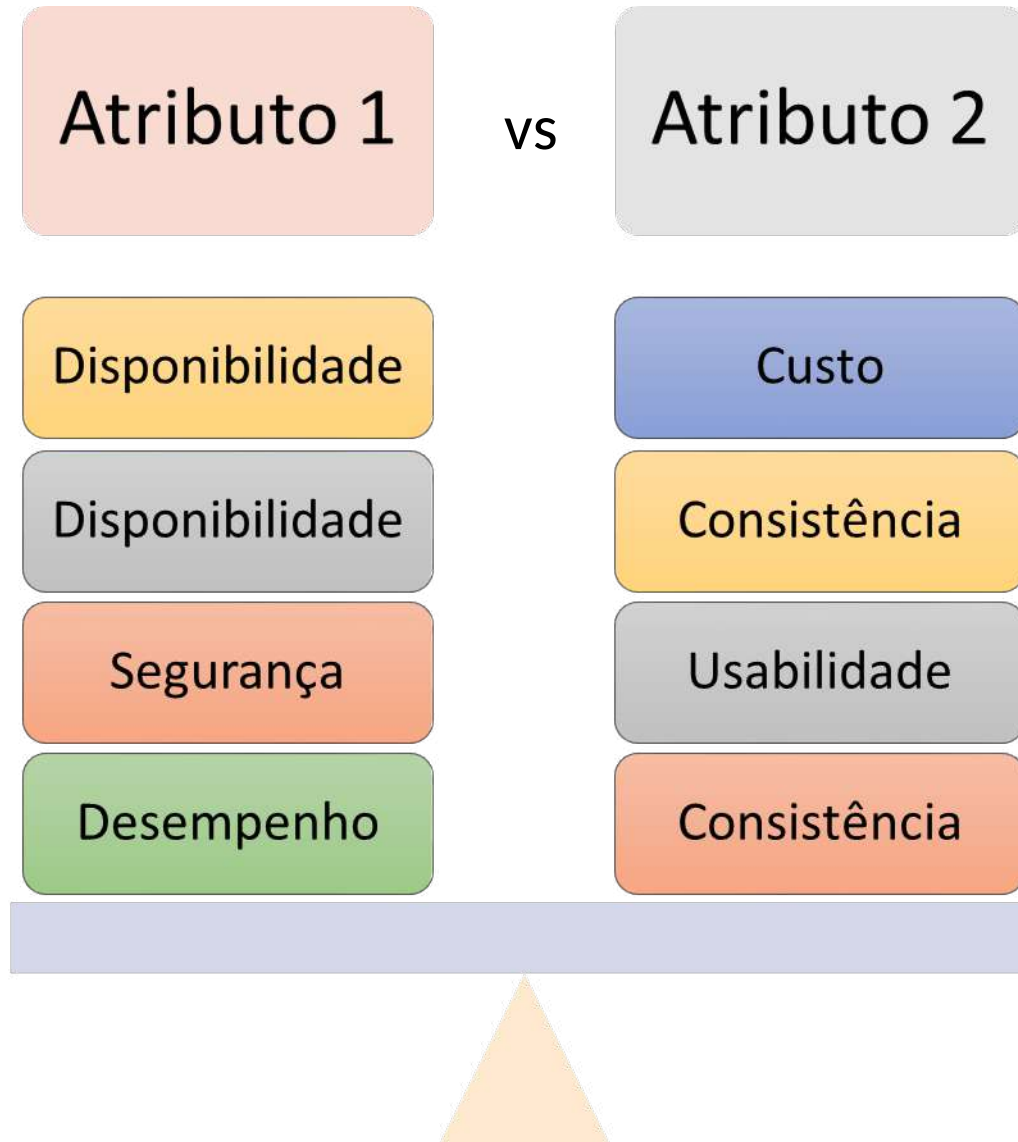
RF5 — Validar ingresso via QR Code

Regra: Impedir validação de ingresso já validado

Concern: O sistema deve funcionar mesmo com internet instável no local do evento



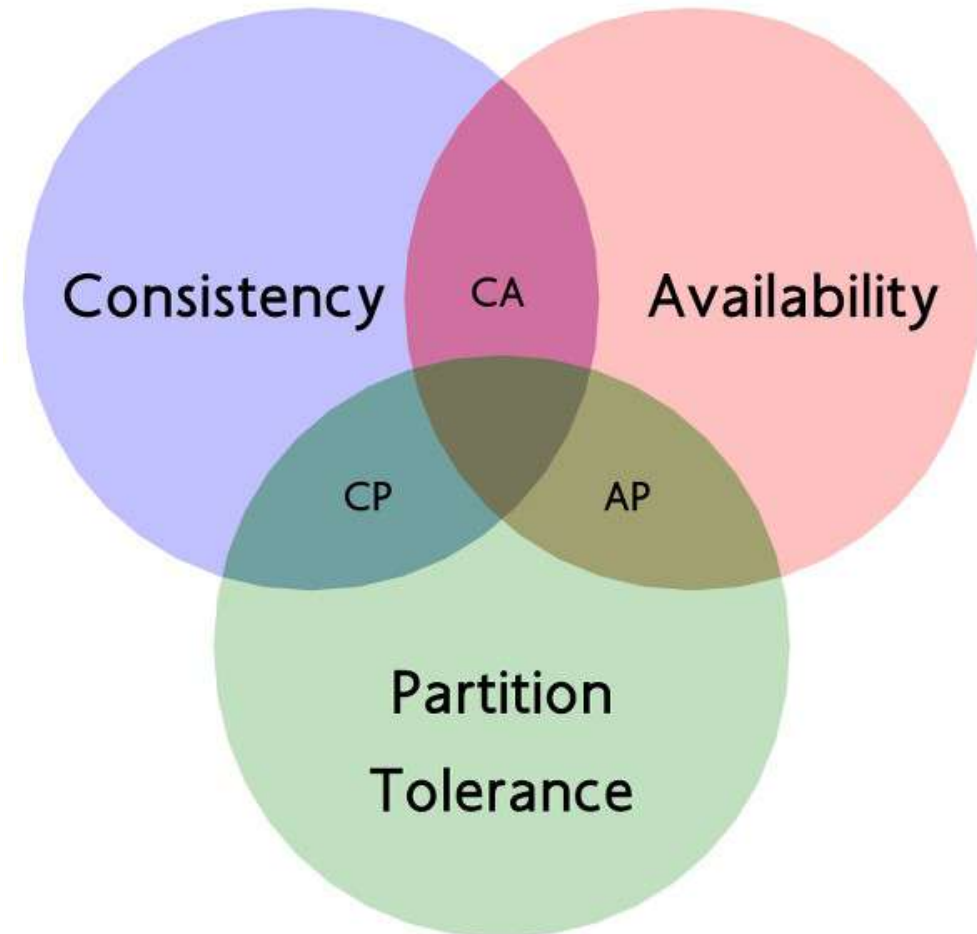
Tipos de *Trade-offs*



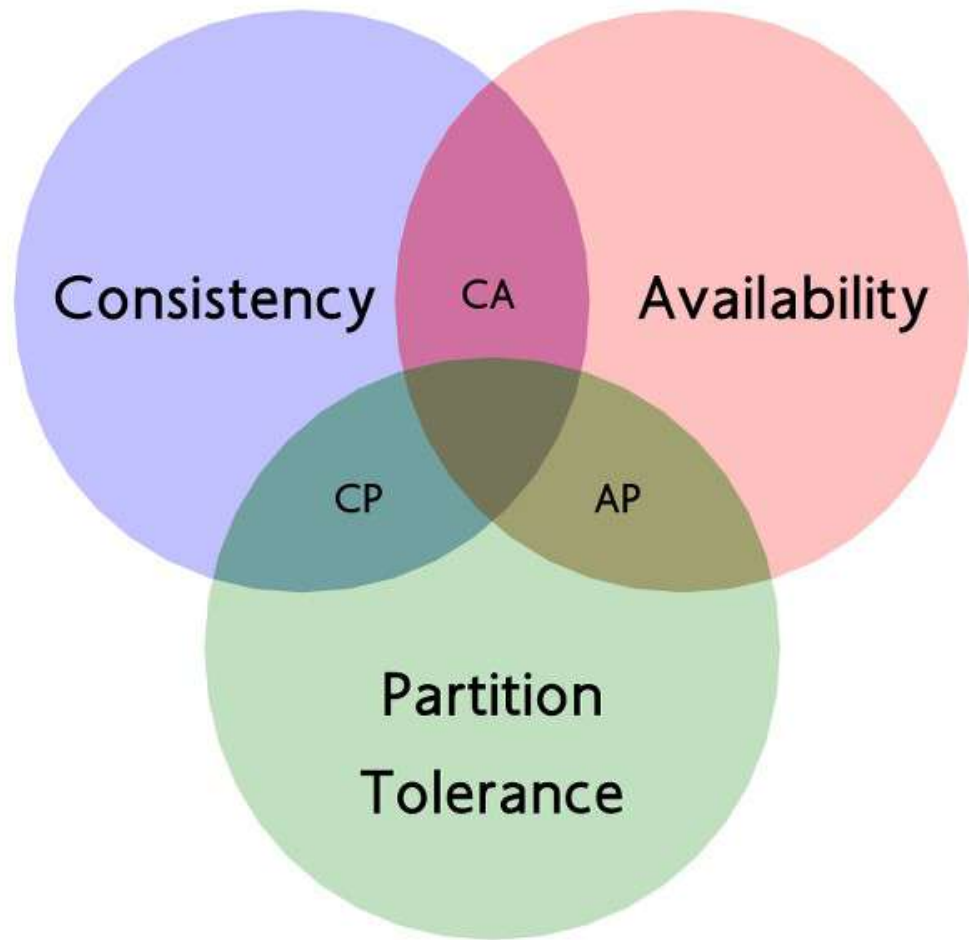
TEOREMA CAP



Eric Allen Brewer é professor emérito de ciência da computação na Universidade da Califórnia, Berkeley e vice-presidente de infraestrutura do Google



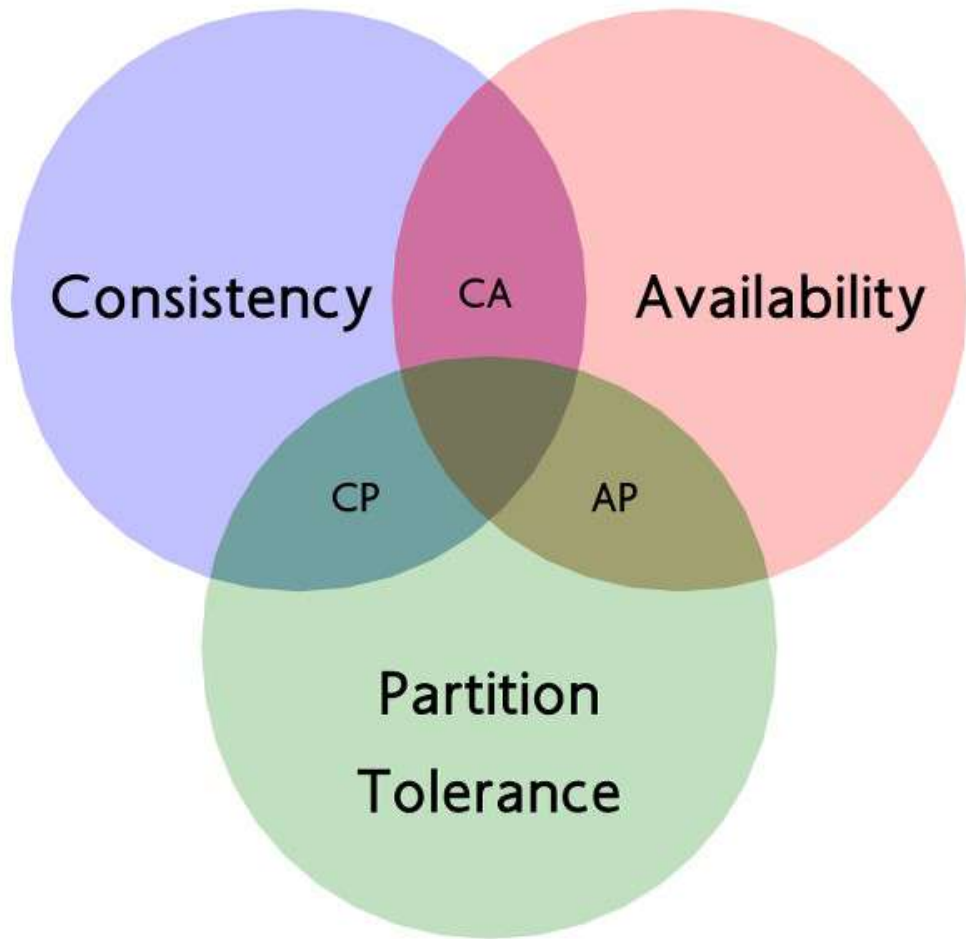
TEOREMA CAP



CA:

- ❑ **Não existe** em Sistemas Distribuídos quando há falha de rede.
- ❑ Só existe em sistemas **sem partição**
 - ❑ Sistemas **não distribuídos**
 - ❑ **Ex:** Monolitos com Base de Dados centralizada

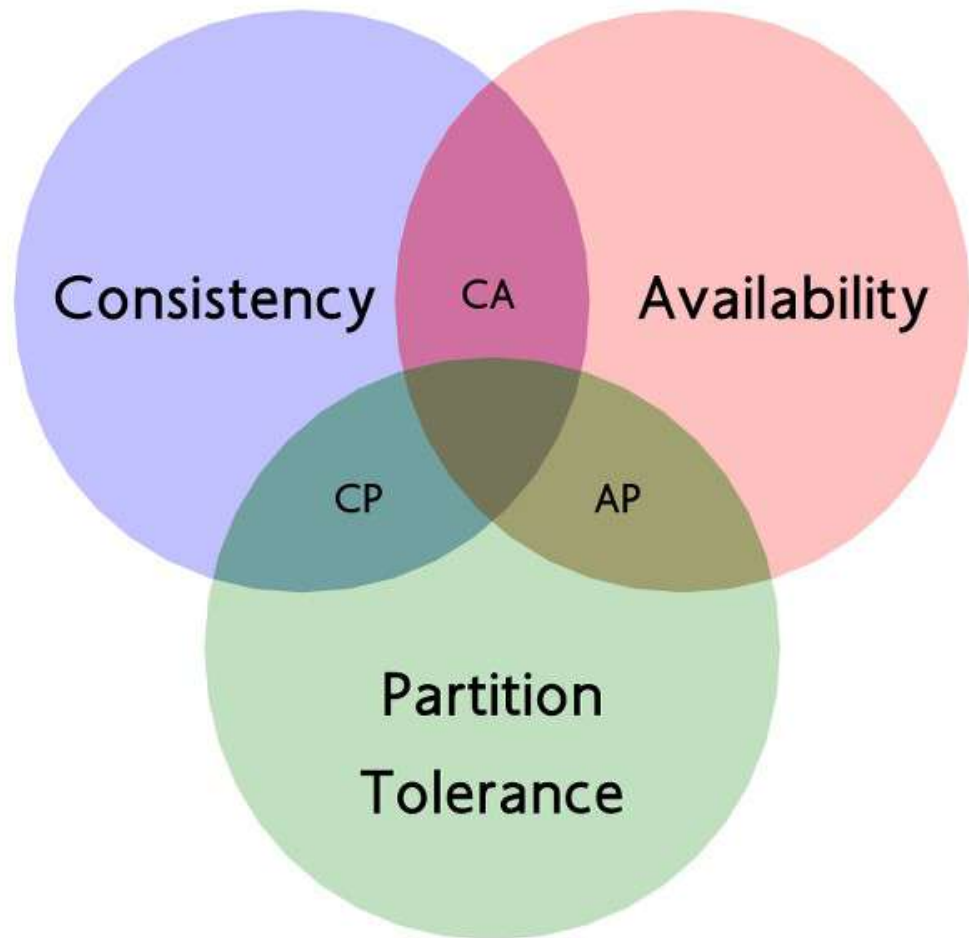
TEOREMA CAP



CP:

- ❑ Mantém dados consistentes
 - ❑ E se houver partição na rede
 - ❑ Nega requisições

TEOREMA CAP

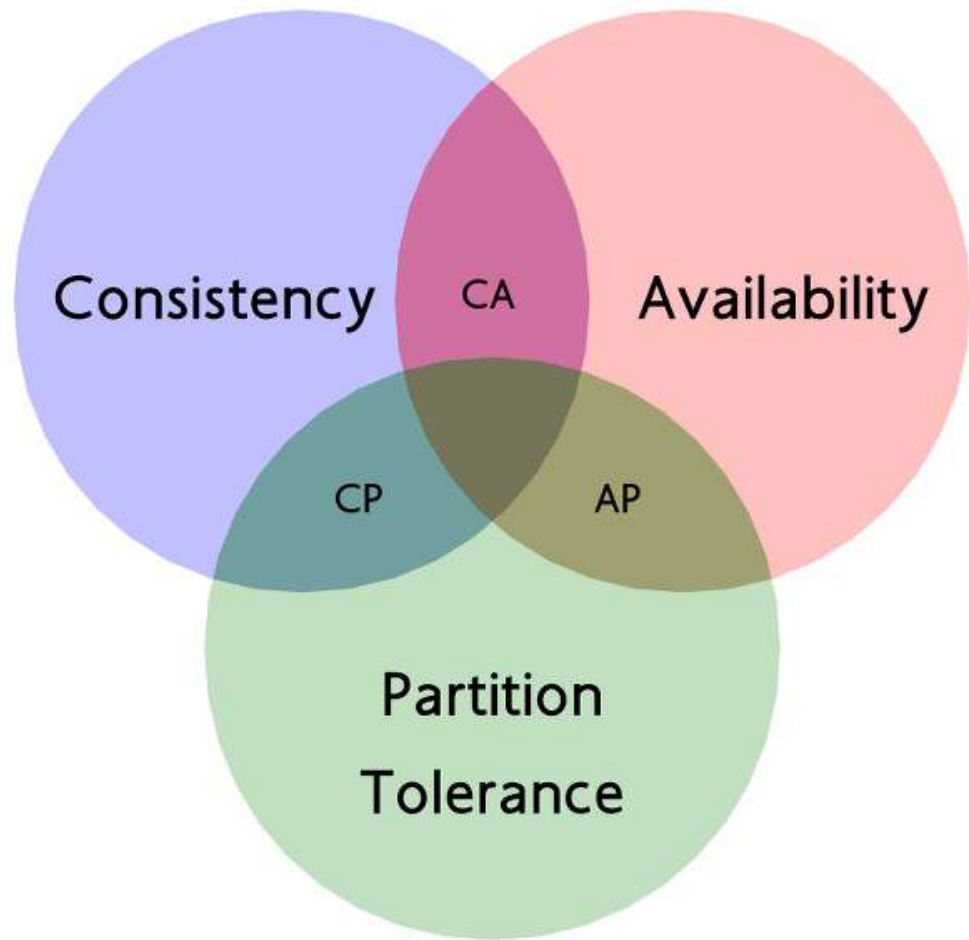


AP:

O sistema continua respondendo a requisições mesmo na presença de partições de rede, porém pode retornar dados temporariamente inconsistentes, garantindo que, eventualmente, todos os nós convergirão para um estado consistente (**consistência eventual**)

- ☐ Disponibilidade mesmo com partição
- ☐ Possibilidade de inconsistência
- ☐ Ideia de consistência eventual

Limitação do TEOREMA CAP

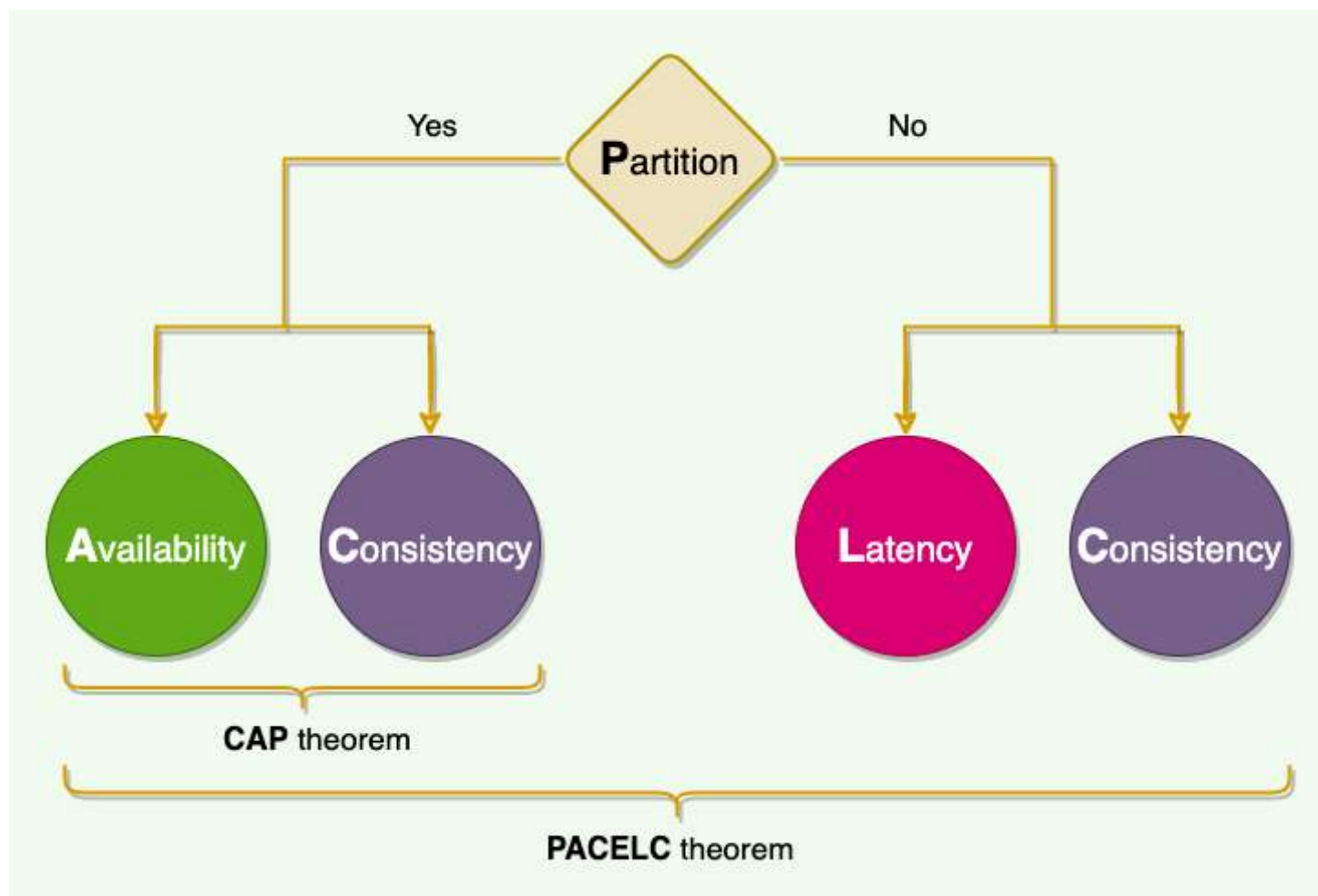


O CAP **não diz nada** sobre o comportamento do sistema em condições normais (sem partição, rede funcionando normalmente e com o sistema saudável)

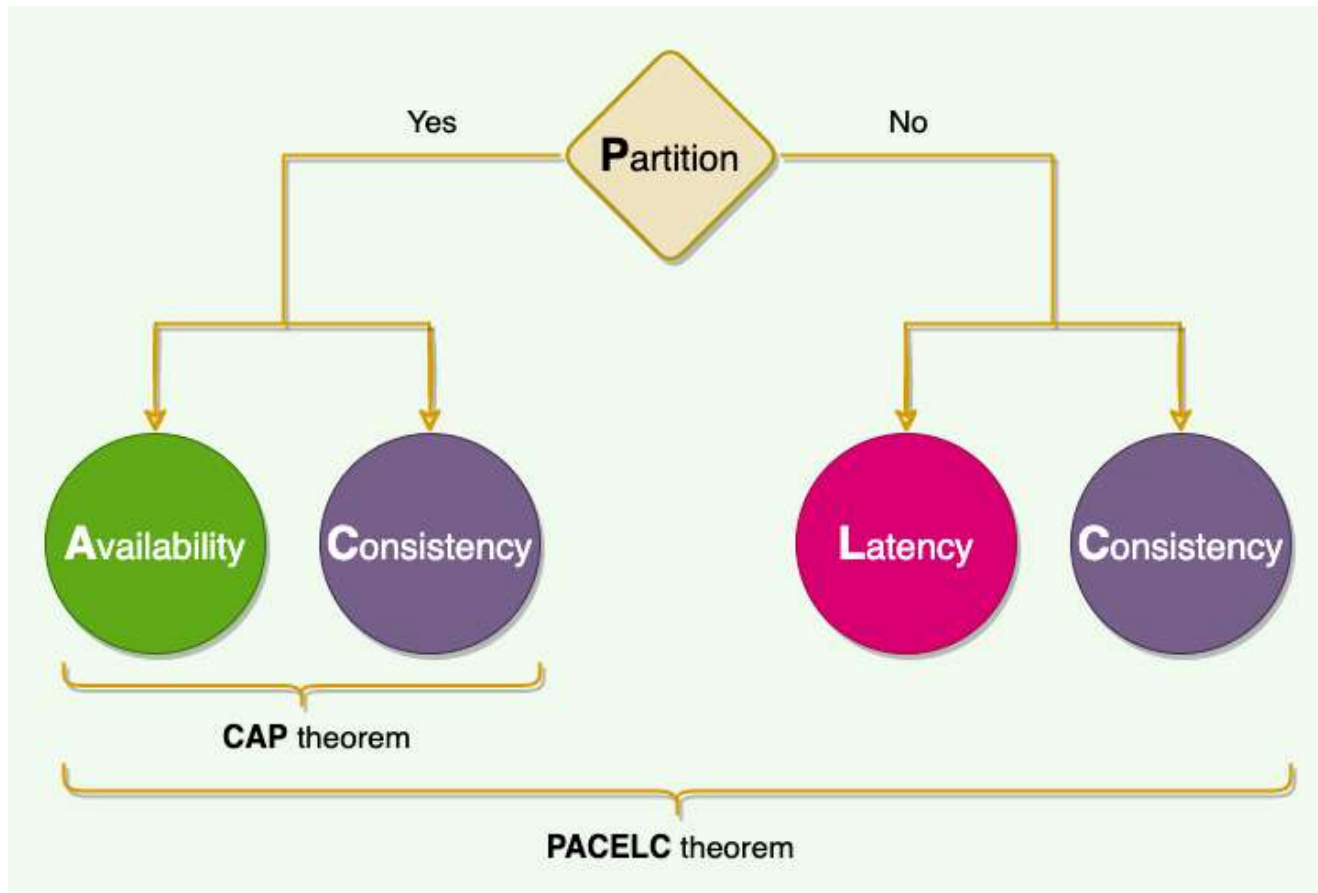
TEOREMA PACELC



Daniel Abadi é professor de Ciência da Computação na Universidade de Maryland, College Park.



TEOREMA PACELC



O problema que o CAP não cobre: No mundo real, a maior parte do tempo:

- ☐ NÃO existe partição
- ☐ O sistema está operando normalmente
- ☐ Precisamos lidar com *trade-offs* mesmo em cenários de sucesso
 - ☐ Latência vs Consistência

TEOREMA PACELC

Imagine um sistema distribuído com múltiplos servidores **sem partição**:

Você pode:

- ☐ Esperar todos sincronizarem
 - ☐ Consistente e Lento
- ☐ Responder rápido com dados locais
 - ☐ Desempenho e Possível Inconsistência
- ☐ Isso **não aparece no teorema CAP**, mas é comum no dia a dia

Trade-offs

NÃO são erros

NÃO são falhas de arquitetura

São escolhas conscientes

O erro é ignorar *trade-offs*
e não documentá-los!

O Papel do Arquiteto de Software

O Arquiteto como mediador de *trade-offs* (compensações)

NÃO EXISTE ARQUITETURA PERFEITA

NÃO EXISTE “RECEITA DE BOLO” PARA ARQUITETURA

Decisões favorecem alguns atributos e prejudicam outros

Logo, o Arquiteto precisa:

- Avaliar impactos
- Justificar escolhas
- Assumir consequências

“Arquitetura é a arte de tomar decisões difíceis diante de restrições reais.” **(Bass e Richards)**

Levantar
Requisitos
com
Stakeholders

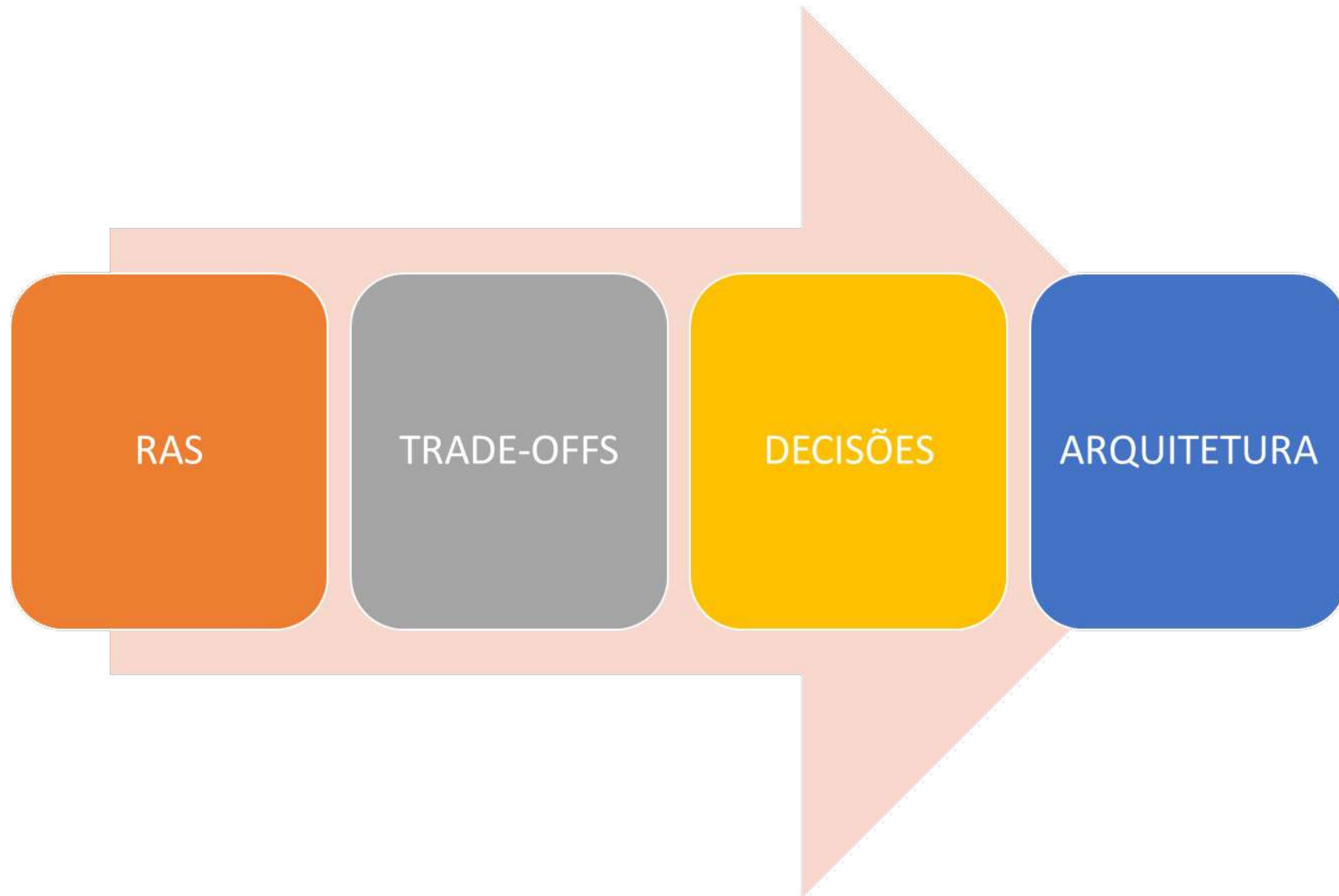
Refinar
Requisitos

Classificar os
RF e RNF

Registrar os
Concerns
Arquiteturais

Filtrar RF e
RNF que
impactam
atributos de
qualidade
da
arquitetura

**Lista de Requisitos
Arquiteturalmente
Significativos
(RAS)**



Conclusão

- ❑ Não existe arquitetura perfeita
- ❑ Toda decisão tem impacto
- ❑ Arquitetura é equilíbrio
- ❑ *Trade-offs* são inevitáveis

O melhor arquiteto não é o que evita *trade-offs*, mas o que faz as melhores escolhas diante deles.